

Projekt Programowanie Współbieżne Wydział Elektrotechniki Automatyki i Informatyki Politechnika Świętokrzyska	
Imię i Nazwisko: Michał Kowalski Krzysztof Wysocki	Grupa dziekańska: 3IZ11A
Temat Projektu: Zarządzania kinem i seansami filmowymi	

1.Opis problemu i cel systemu

Kino jako przedsiębiorstwo usługowe musi sprawnie zarządzać seansami filmowymi, sprzedając biletów i harmonogramem sal projekcyjnych. Tradycyjne metody oparte na papierowych rezerwacjach lub prostych arkuszach kalkulacyjnych nie zapewniają wymaganej wydajności, bezpieczeństwa danych ani skalowalności.

Celem projektu jest zaprojektowanie i implementacja systemu informatycznego do zarządzania kinem, który umożliwia:

- zarządzanie repertuarem filmowym i seansami przez menadżera,
- sprzedaż i zwrot biletów przez pracowników i klientów,
- automatyczne wykrywanie i blokowanie konfliktów terminów w salach kinowych,
- trwałe przechowywanie danych w relacyjnej bazie danych MySQL,
- czytelną architekturę obiektową z zastosowaniem wzorców projektowych.

Słowniki pojęć

Pojęcie	Definicja
Film	Materiał filmowy dostępny w repertuarze kina. Zawiera tytuł i czas trwania w minutach.
Sala (Hall)	Pomieszczenie kinowe o określonej pojemności, w którym odbywają się seanse.
Miejsce (Seat)	Pojedyncze siedzenie w sali. Rząd 1 oznacza miejsca premium. Stan: wolne lub zajęte.
Seans (Screening)	Emisja konkretnego filmu w konkretnej sali w określonym przedziale czasowym. Centralna klasa modelu.
Bilet (Ticket)	Obiekt upoważniający klienta do wejścia na seans. Status: PURCHASED, RESERVED lub RETURNED.
Klient	Użytkownik kupujący i zwracający bilety. Przechowuje listę własnych biletów.
Pracownik	Kasjer obsługujący sprzedaż biletów przy kasie.
Menadżer	Osoba ustalająca repertuar filmowy i zarządzająca seansami.

2. Zakres projektu

Elementy objęte w projekcie

- Model domenowy: Film, Hall, Seat, Ticket, User (Customer, Employee, Manager), Screening
- Logika biznesowa: wykrywanie konfliktów sal, rezerwacja miejsc, zakup i zwrot biletów
- Wzorzec Builder (ScreeningBuilder) – walidacja i budowanie obiektów Screening
- Wzorzec Fasada (CinemaSystemFacade) – uproszczone API do głównych operacji
- Warstwa persystencji JDBC: ScreeningRepository, TicketRepository, UserRepository
- Baza danych MySQL: 6 tabel, dane testowe
- Testy jednostkowe JUnit 5: 29 testów pokrywających model, wzorce i logikę

3. Zakres projektu

Klient

- WF-K01: Klient może zalogować się do systemu podając login i hasło.
- WF-K02: Klient może przeglądać listę dostępnych seansów z wolnymi miejscami.
- WF-K03: Klient może zakupić bilet na wybrany seans i miejsce.
- WF-K04: Klient może zwrócić zakupiony bilet.

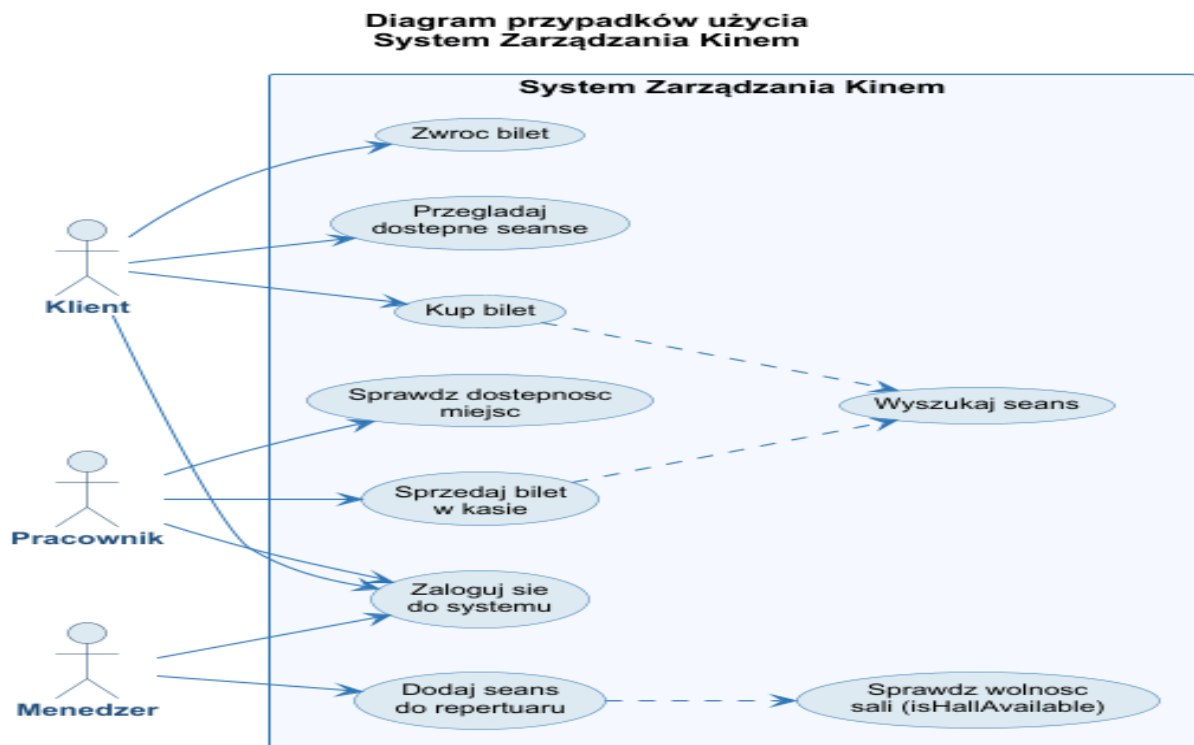
Pracownik

- WF-P01: Pracownik może zalogować się do systemu.
- WF-P02: Pracownik może sprzedać bilet klientowi w kasie.
- WF-P03: Pracownik może sprawdzić dostępność miejsc na dany seans.

Menadżer

- WF-M01: Menadżer może zalogować się do systemu.
- WF-M02: Menadżer może dodać nowy seans do repertuaru.
- WF-M03: System automatycznie blokuje dodanie seansu kolidującego z istniejącym w tej samej sali.

4. Diagram przypadków użycia



Aktor	Przypadek użycia	Opis
Wszyscy trzej	UC-00: Zaloguj się do systemu	Wspólny UC – uwierzytelnienie przez login i hasło (CinemaSystem.authenticate)
Klient	UC-01: Przeglądaj dostępne seanse	Lista seansów z checkAvailability() == true
Klient	UC-02: Kup bilet	Wybór seansu i miejsca, zakup przez CinemaSystemFacade.buyTicket()
Klient	UC-03: Zwroc bilet	Zwrot przez CinemaSystemFacade.refundTicket(), zwolnienie miejsca
Pracownik	UC-04: Sprzedaj bilet w kasie	Sprzedaż biletu klientowi przy kasie
Pracownik	UC-05: Sprawdź dostępność miejsc	Podgląd wolnych miejsc na wybrany seans
Menadżer	UC-06: Dodaj seans do repertuaru	Tworzenie seansu przez ScreeningBuilder – walidacja + zapis

Relacje między przypadkami użycia:

- UC-02 <<include>> wyszukaj seans – zakup wymaga wybrania seansu
- UC-04 <<include>> wyszukaj seans – sprzedaż w kasie wymaga wyboru seansu
- UC-06 <<include>> sprawdź wolność sali – Builder wywołuje isHallAvailable() w build()

5. Wymagania niefunkcjonalne

ID	Kategoria	Wymaganie
WNF-01	Integralność	Zakup biletu w transakcji ACID (SELECT FOR UPDATE w MySQL). Rollback przy błędzie.
WNF-02	Integralność	Konflikty terminów sal wykrywane w isConflictWith() i isHallAvailable() – dwa poziomy ochrony.
WNF-03	Bezpieczeństwo	Hasła przechowywane zahashowane w produkcji (BCrypt). Wersja demo: plain text.
WNF-04	Testowalność	Stan CinemaSystem resetowany między testami. 38 testów, pokrycie $\geq 50\%$.
WNF-05	Konserwacja	Zasada SRP: każda klasa ma jedną odpowiedzialność. Pakiety separują warstwy.
WNF-06	Technologia	Java 17+, MySQL 8+, Maven 3.8+, JUnit 5. Bez zewnętrznych frameworków biznesowych.
WNF-07	Niezawodność	Rollback transakcji SQL gwarantuje brak niespójnych danych.
WNF-08	Czytelność	Builder eliminuje wieloparametrowe konstruktory. Fasada ukrywa złożoność operacji.

6. Opis architektury systemu

Warstwa aplikacji

Warstwa	Pakiet Java	Zawartość
Model domenowy	cinema.model	Film, Hall, Seat, Ticket, TicketStatus, User(abstrakt), Customer, Employee, Manager, Screening
Logika systemu	cinema.service	CinemaSystem (Singleton) – centralny rejestr seansów i użytkowników z logiką walidacji
Wzorzec Builder	cinema.builder	ScreeningBuilder – budowanie Screening z walidacją i rejestracją w CinemaSystem
Wzorzec Fasada	cinema.facade	CinemaSystemFacade – uproszczone API: buyTicket, refundTicket, createScreening
Persystencja	cinema.db	DatabaseConnection, ScreeningRepository, TicketRepository, UserRepository

Schemat bazy danych MySQL

Schemat bazy danych

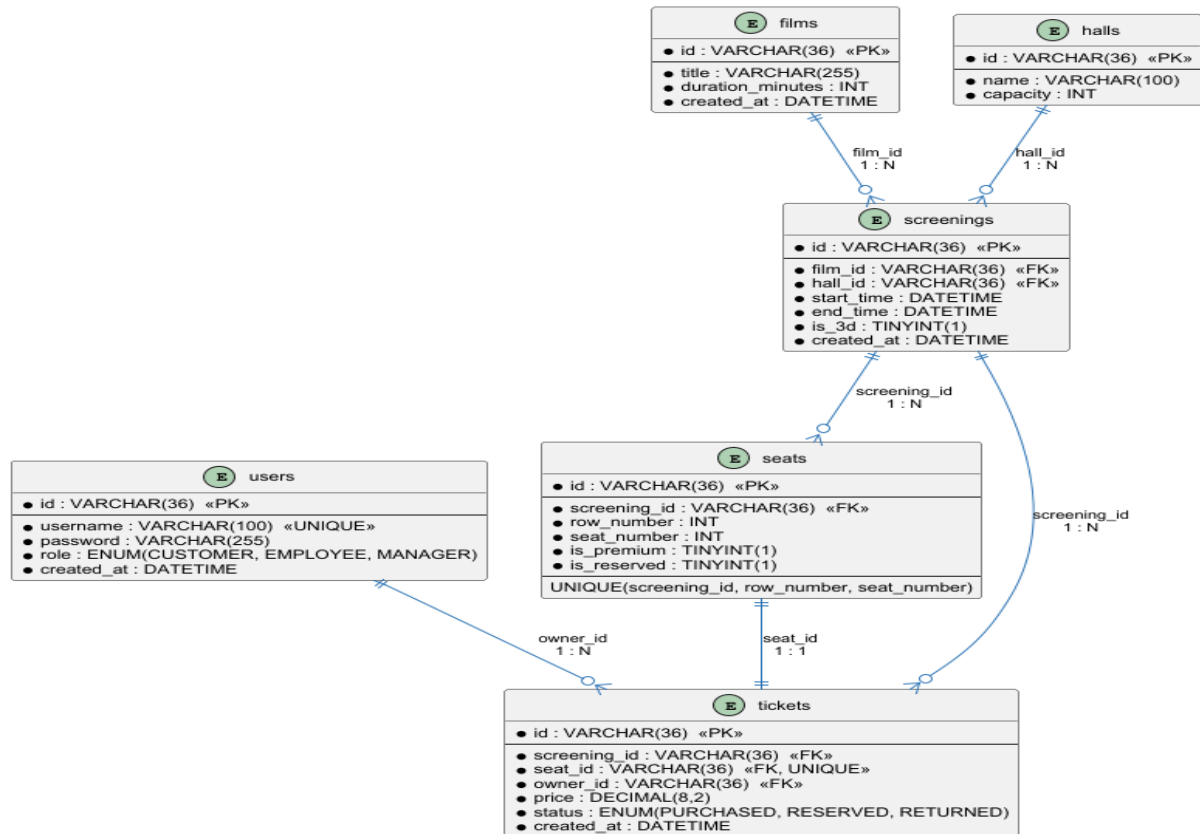


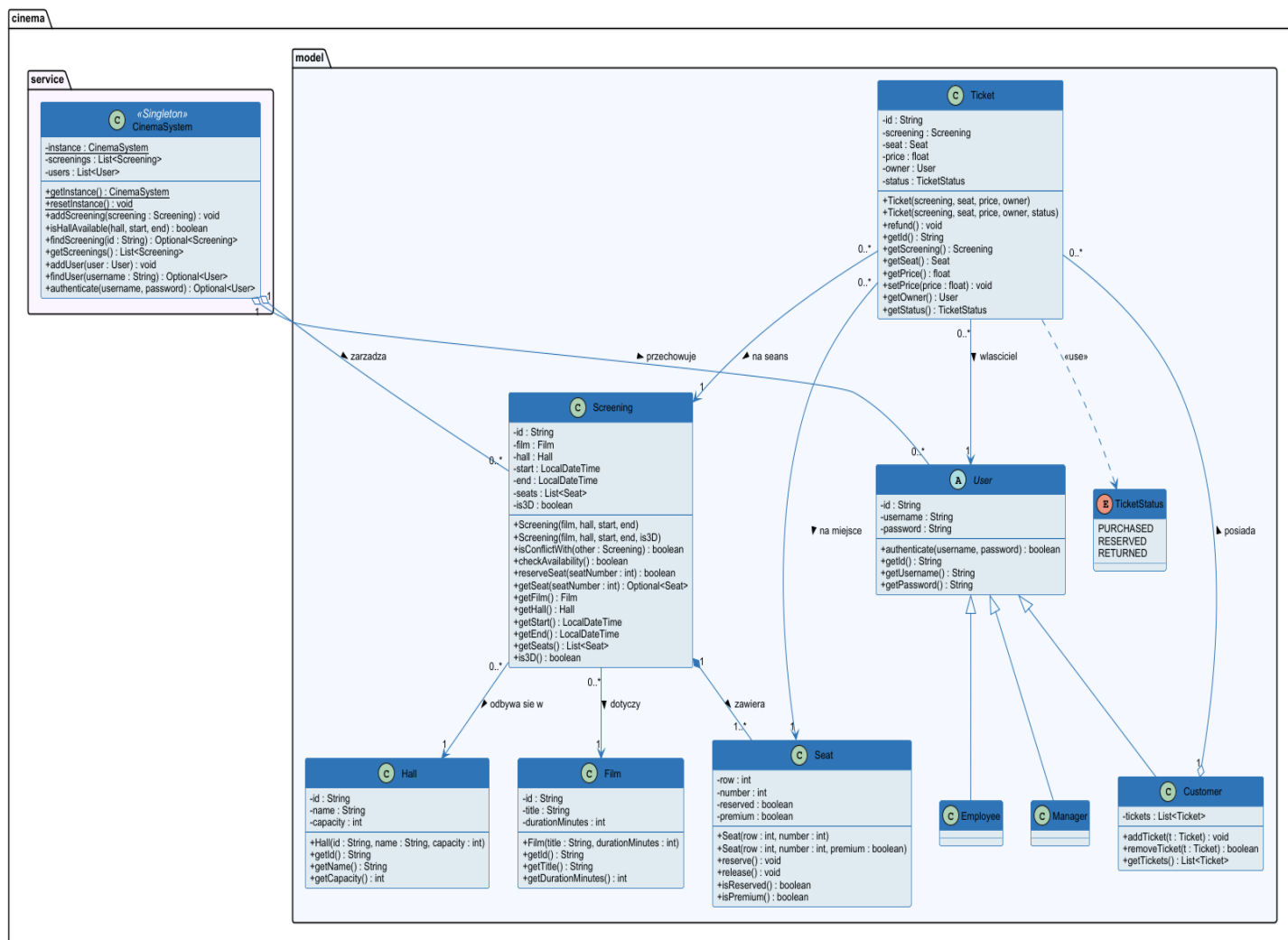
Tabela	Kluczowe kolumny	FK / ograniczenia
users	id PK, username UNIQUE, password, role ENUM(CUSTOMER,EMPLOYEE,MANAGER)	Brak
films	id PK, title, duration_minutes INT	Brak
halls	id PK, name, capacity INT	Brak
screenings	id PK, film_id, hall_id, start_time, end_time, is_3d TINYINT	FK→films, FK→halls
seats	id PK, screening_id, row_number, seat_number, is_premium, is_reserved	FK→screenings (CASCADE), UNIQUE(screening_id,row,seat)
tickets	id PK, screening_id, seat_id UNIQUE, owner_id, price DECIMAL, status ENUM	FK→screenings, FK→seats, FK→users

Relacje

- films → screenings (1:N) | halls → screenings (1:N)
- screenings → seats (1:N) – miejsca generowane automatycznie przy tworzeniu seansu
- screenings → tickets (1:N) | seats → tickets (1:1, UNIQUE na seat_id)
- users → tickets (1:N)

7. Diagram klas

Diagram klas - Model domenowy
cinema.model + cinema.service



Cinema.model

Klasa	Pola	Kluczowe metody
Film	id, title, durationMinutes	getId(), getTitle(), getDurationMinutes()
Hall	id, name, capacity	getId(), getName(), getCapacity()
Seat	row, number, reserved, premium	reserve(), release(), isReserved(), isPremium()
User (abstract)	id, username, password	authenticate(u,p):boolean

Customer : User	tickets: List<Ticket>	addTicket(), removeTicket(), getTickets()
Employee : User	–	–
Manager : User	–	–
TicketStatus	enum: PURCHASED, RESERVED, RETURNED	–
Ticket	id, screening, seat, price, owner, status	refund(), getStatus(), getSeat(), getOwner()
Screening	id, film, hall, start, end, seats, is3D	isConflictWith(), checkAvailability(), reserveSeat(n), getSeat(n)

Cinema.service

Klasa	Pakiet	Kluczowe metody
CinemaSystem (Singleton)	cinema.service	getInstance(), addScreening(), isHallAvailable(), authenticate(), addUser()
ScreeningBuilder (Builder)	cinema.builder	setFilm/Hall/Start/End/3D(), setEndFromFilm(), build()
CinemaSystemFacade (Fasada)	cinema.facade	buyTicket(), refundTicket(), createScreening(), getAvailableScreenings()

Cinbema.db

Klasa	Kluczowe metody
DatabaseConnection	getInstance(), getConnection(), close()
ScreeningRepository	findAll(), findAvailable(), save(), isHallAvailable()
TicketRepository	getAvailableSeats(), buyTicket(), refundTicket()
UserRepository	authenticate(), findById(), register(), findAll()

Relacje w diagramie klas

- Dziedziczenie (pusty grot ▷): Customer, Employee, Manager → User (abstract)
- Kompozycja (■--): Screening *-- Seat (seans zawiera miejsca, generowane w konstruktorze)
- Agregacja (◇--): Customer o-- Ticket | CinemaSystem o-- Screening | CinemaSystem o-- User
- Asocjacja (→): Ticket → Screening, Seat, User | Screening → Film, Hall
- Zależność (--▷): CinemaSystemFacade → ScreeningBuilder | Repozytoria → DatabaseConnection

8. Opis głównych przypadków użycia i scenariuszy

Kup bilet

Aktorzy	Klient
Warunki wstępne	Klient zalogowany; istnieje seans z wolnymi miejscami
Warunki końcowe	Bilet PURCHASED przypisany do klienta; miejsce is_reserved = true

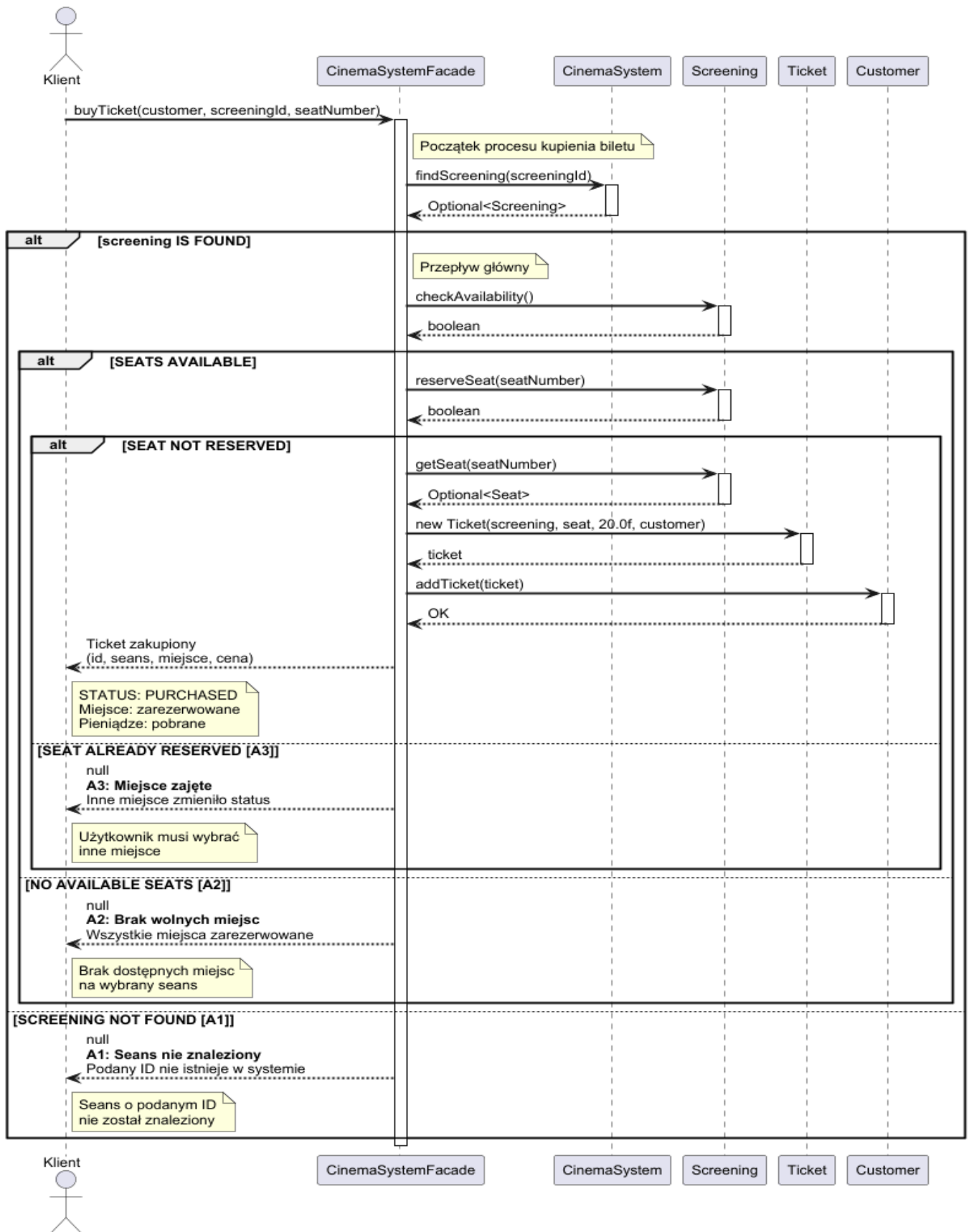
Przeptyw główny:

1. Klient pobiera dostępne seanse (`facade.getAvailableScreenings()`).
2. System zwraca seanse gdzie `checkAvailability() == true`.
3. Klient podaje ID seansu i numer miejsca.
4. Fasada: `findScreening()` → `checkAvailability()` → `reserveSeat(n)` → `new Ticket(status=PURCHASED)` → `customer.addTicket()`.
5. Fasada zwraca Ticket – potwierdzenie zakupu.

Przeptywy alternatywne:

- Krok 2: brak seansów z wolnymi miejscami → pusta lista.
- Krok 4: `reserveSeat() == false` (zajęte) → fasada zwraca null, klient wybiera inne miejsce.
- Krok 4: seans nie istnieje w systemie → fasada zwraca null.

UC: KUP BILET Przepływ główny + alternatywny



Zwróć bilet

Aktorzy	Klient
Warunki wstępne	Klient zalogowany; posiada bilet PURCHASED lub RESERVED
Warunki końcowe	Status biletu RETURNED; miejsce is_reserved = false; bilet usunięty z listy klienta

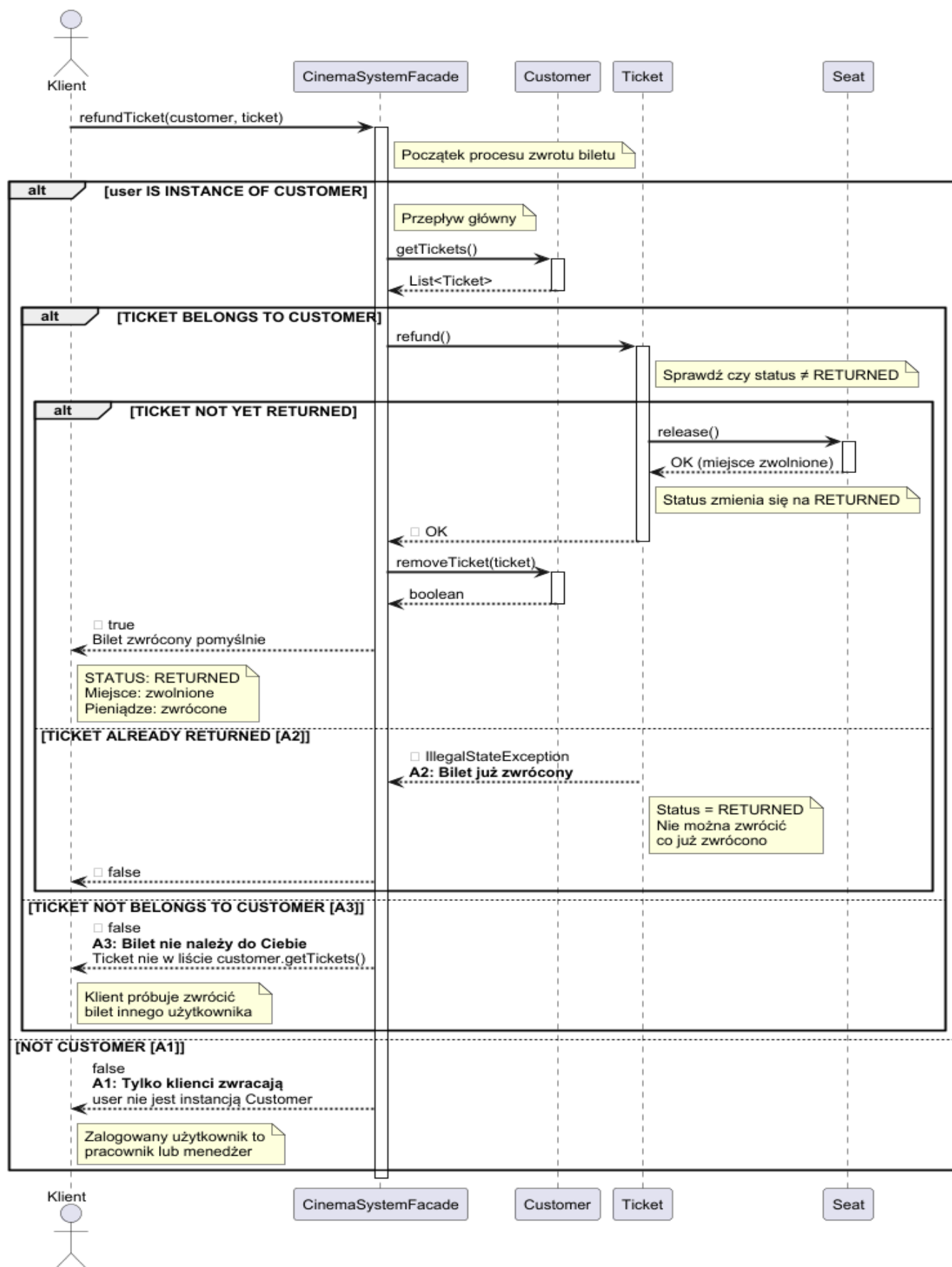
Przeptyw główny:

6. Klient wywołuje `facade.refundTicket(customer, ticket)`.
7. Fasada sprawdza `customer.getTickets().contains(ticket)`.
8. Fasada wywołuje `ticket.refund()` → `seat.release()` + `status = RETURNED`.
9. Fasada wywołuje `customer.removeTicket(ticket)`.
10. Fasada zwraca `true` – potwierdzenie zwrotu.

Przeptywy alternatywne:

- Krok 2: bilet nie należy do klienta → fasada zwraca `false`.
- Krok 3: bilet już zwrócony → `IllegalStateException`.

UC: ZWRÓĆ BILET Przeływ główny + alternatywny



9. Opis głównych przypadków użycia i scenariuszy

W planowanej rozbudowie serwer TCP nasłuchuje na porcie 9000. Każde połączenie klienta obsługiwane jest przez osobny wątek (ExecutorService). Wiadomości przesyłane jako serializowane obiekty Java klasy Message.

Typ wiadomości	Kierunek	Payload / opis
LOGIN	Klient → Serwer	String[] {username, password}
LOGIN_RESPONSE	Serwer → Klient	"OK:ROLA" lub "FAIL"
GET_SCREENINGS	Klient → Serwer	null – żądanie listy seansów
SCREENINGS_RESPONSE	Serwer → Klient	List<String> – opisy dostępnych seansów
BUY_TICKET	Klient → Serwer	String[] {screeningId, seatNumber}
BUY_TICKET_RESPONSE	Serwer → Klient	"OK:ticketId" lub "FAIL:powód"
REFUND_TICKET	Klient → Serwer	String ticketId
REFUND_TICKET_RESPONSE	Serwer → Klient	"OK" lub "FAIL:powód"
ERROR	Serwer → Klient	String opis błędu

10. Opis wykorzystania współbieżności

Transakcje bazodanowe jako ochrona przed race conditio

TicketRepository.buyTicket() używa SELECT FOR UPDATE – blokuje wiersz miejsca na czas transakcji. Eliminuje sytuację, w której dwóch klientów kupuje to samo miejsce jednocześnie. Rollback przy każdym błędzie SQL.

CinemaSystem – poziom aplikacji

CinemaSystem.isHallAvailable() i addScreening() stanowią dwupoziomową ochronę przed konfliktami. W wersji wielowątkowej metody te powinny być oznaczone jako synchronized lub korzystać z CopyOnWriteArrayList.

Plan na wersję wielowątkową

- CinemaServer – ExecutorService.newCachedThreadPool(), każde połączenie w osobnym wątku.
- CinemaSystem – double-checked locking w Singletonie (volatile + synchronized).
- Listy screenings i users jako CopyOnWriteArrayList.

11. Opis wykorzystania współbieżności

Wzorzec	Kategoria	Klasa / pakiet	Uzasadnienie zastosowania
Singleton	Kreacyjny	CinemaSystem cinema.service	Jeden centralny rejestr seansów i użytkowników – stan musi być spójny w całej aplikacji.
Builder	Kreacyjny	ScreeningBuilder cinema.builder	Seans ma wiele pól; walidacja (sala wolna, end > start) musi nastąpić przed stworzeniem obiektu.
Fasada	Strukturalny	CinemaSystemFacade cinema.facade	Zakup biletu wymaga 5+ kroków na różnych obiektach; fasada enkapsuluje sekwencję w jednym wywołaniu.

Singleton – CinemaSystem

Wzorzec Singleton gwarantuje, że w całej aplikacji istnieje tylko jedna instancja CinemaSystem, przechowująca listę seansów i użytkowników. Metoda getInstance() tworzy instancję przy pierwszym wywołaniu i zwraca tę samą przy kolejnych wywołaniach.

Builder – ScreeningBuilder

Problem: konstruktor Screening z wieloma parametrami jest nieczytelny i podatny na błędy kolejności argumentów. ScreeningBuilder udostępnia czytelne metody łańcuchowe. Metoda setEndFromFilm() automatycznie oblicza czas zakończenia na podstawie durationMinutes. Metoda build() wykonuje kompletną walidację przed stworzeniem obiektu.

Fasada – CinemaSystemFacade

Problem: zakup biletu wymaga kolejno: findScreening → checkAvailability → reserveSeat → new Ticket → addTicket. Fasada enkapsuluje całą sekwencję w metodzie buyTicket(user, screeningId, seatNumber), którą klient wywołuje bez znajomości szczegółów implementacji.

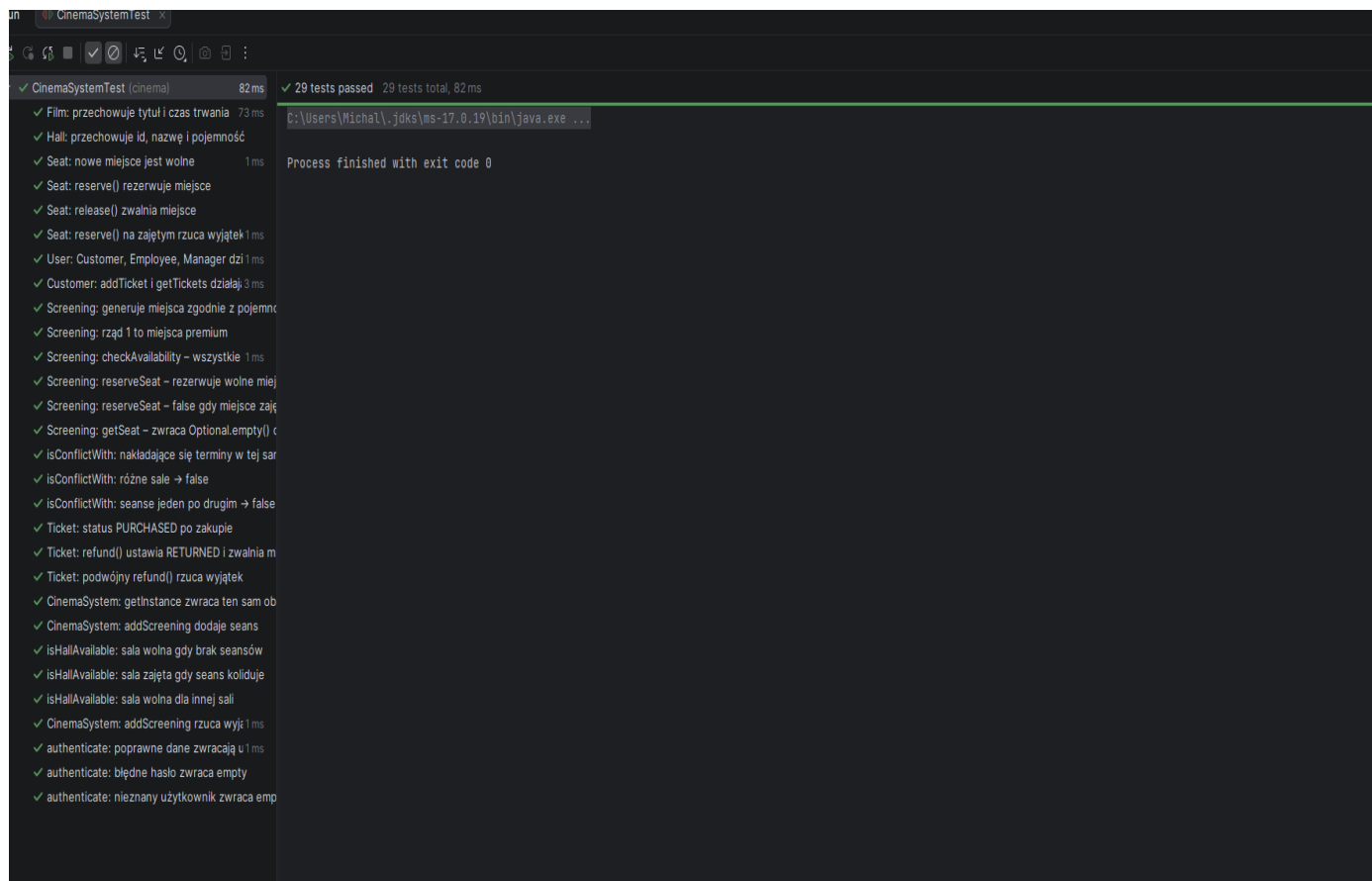
12. Opis testów

Framework i organizacja testów

Testy jednostkowe napisane w JUnit 5 (Jupiter). CinemaSystem resetowany przed każdym testem przez CinemaSystem.resetInstance() w metodzie @BeforeEach. Testy nie wymagają uruchomionej bazy danych – operują wyłącznie na stanie in-memory.

CinemaSystemTest.java

Testy	Zakres	Co weryfikują
1–2	Film, Hall	Przechowywanie atrybutów, nie-null id
3–6	Seat	reserve(), release(), double reserve → IllegalStateException
7–8	User, Customer	Dziedziczenie assertInstanceOf, addTicket/getTickets
9–14	Screening	Liczba wygenerowanych miejsc, checkAvailability, reserveSeat, getSeat, releaseSeat
15–17	isConflictWith	Nakładanie w tej samej sali (true), różna sala (false), terminy sekwencyjne (false)
18–20	Ticket	Status PURCHASED, refund() → RETURNED + miejsce wolne, double refund → wyjątek
21–22	CinemaSystem – Singleton	assertSame, addScreening → size++
23–25	isHallAvailable	Wolna (true), zajęta (false), inna sala (true)
26	addScreening – konflikt	IllegalStateException przy konflikcie
27–29	authenticate	Poprawne dane (present), złe hasło (empty), nieznany user (empty)



13. Prezentacja programu użytkownika

Klasa Main.java prezentuje pełny przepływ operacji wypisując wyniki do konsoli

```
=== Użytkownicy ===
Manager{id='5e23aeb5-af8a-4e3f-89ca-9f0ba80c6cb9', username='admin'}
Employee{id='dcf1c18a-9d53-44b5-911a-023915e6d868', username='kasjer1'}
Customer{id='059b3022-31b5-41be-ba34-cdda25e7f936', username='jan.kowalski'}

=== Logowanie ===
OK: Customer{id='059b3022-31b5-41be-ba34-cdda25e7f936', username='jan.kowalski'}
FAIL: błędne hasło.

=== Seanse ===
Screening{id='7596ede7-28ed-4c9b-88eb-c41f19e4d80a', film='Inception', hall='Sala 1', start=2026-05-10T18:00, end=2026-05-10T20:28, 3D=false}
Screening{id='d24ad227-8066-4f8b-9303-51a5a3885bae', film='Interstellar', hall='Sala 2', start=2026-05-10T18:00, end=2026-05-10T20:49, 3D=false}

=== Dostępność sali ===
Sala 1 wolna 19:00-21:00? false

=== Konflikt terminów ===
s1 koliduje z seansem o 19:00 w Sali 1? true
s2 koliduje z seansem o 19:00 w Sali 1? false
Zablokowano dodanie: Sala 'Sala 1' jest już zajęta w podanym terminie.

=== Zakup biletu ===
Dostępność s1: true
Rezerwacja miejsca 5: true
Bilet: Ticket{id='16f60d1e-dec3-4be2-8f38-0257704493a7', film='Inception', seat=Seat{row=1, number=5, reserved=true, premium=true}, price=20.0, status=PURCHASED}
Bilety Klienta: 1

=== Zwrot biletu ===
Status: RETURNED
Miejsce 5 wolne: true

Process finished with exit code 0
|
```

14. Prezentacja programu użytkownika

Osoba	Zrealizowane elementy
Michał Kowalski	Klasy modelu: Film, Hall, Seat, Ticket (TicketStatus), User (abstrakt), Customer, Employee, Manager. Klasa Screening: pola film/hall/start/end/seats/is3D, metody isConflictWith(), checkAvailability(), reserveSeat(), getSeat(). Diagram klas modelu domenowego. ScreeningBuilder (cinema.builder): setFilm/Hall/Start/End/3D(), setEndFromFilm(), build() z walidacją. CinemaSystemFacade (cinema.facade): buyTicket(), refundTicket(), createScreening(), getAvailableScreenings(). Testy JUnit 5: Builder ×3, Fasada ×3. Scenariusze UC-02, UC-03.
Krzysztof Wysocki	CinemaSystem (Singleton): addScreening(), isHallAvailable(), authenticate(), addUser(). Konfiguracja Maven (pom.xml z JUnit 5 + MySQL Connector). Schemat bazy MySQL: 6 tabel, dane testowe (3 filmy, 2 sale, 5 użytkowników, 3 seanse). Diagram przypadków użycia. DatabaseConnection (Singleton JDBC). ScreeningRepository: findAll(), findAvailable(), save() (transakcja), isHallAvailable(). TicketRepository: getAvailableSeats(), buyTicket() (SELECT FOR UPDATE), refundTicket() (transakcja). UserRepository: authenticate(), findById(), register(). Testy JUnit 5: CinemaSystem ×3.
Obaj	Integracja kodu, wzajemny przegląd, testy jednostkowe, dokumentacja projektowa.

15. Instrukcja uruchomienia projektu

Wymagania systemowe

- Java 17 lub nowsza (JDK – nie tylko JRE). Weryfikacja: `java -version`
- Apache Maven 3.8+. Weryfikacja: `mvn -version`
- MySQL 8.0+ działający lokalnie na porcie 3306
- Git

Klonowanie repozytorium

```
git clone https://github.com/[repo]/cinema-management.git
```

```
cd cinema-management
```

Inicjalizacja bazy danych

```
mysql -u root -p < cinema_km1.sql
```

Skrypt tworzy bazę `cinema_db` z 6 tabelami i danymi testowymi: 5 użytkowników, 3 filmy, 2 sale, 3 seanse, 1 przykładowy bilet.

Konfiguracja połączenia JDBC

Edycja `src/main/java/cinema/db/DatabaseConnection.java`:

```
private static final String USER = "root"; // ← twój login MySQL
```

```
private static final String PASSWORD = ""; // ← twoje hasło MySQL
```

Kompilacja i uruchomienie testów

W katalogu głównym projektu należy wykonać:

```
mvn clean test
```

Prawidłowe wykonanie kończy się komunikatem `BUILD SUCCESS`. Testy jednostkowe nie wymagają uruchomionej bazy danych.

Budowanie i uruchomienie programu

Aby zbudować plik JAR i uruchomić główny scenariusz demonstracyjny z klasy `cinema.Main`, należy wykonać:

```
mvn clean package
```

```
java -jar target/cinema-km1.jar
```


Klasa Main prezentuje logowanie użytkowników, dodawanie seansów, wykrywanie konfliktów terminów, rezerwację miejsca, zakup biletu oraz jego zwrot. Ten scenariusz działa bez połączenia z MySQL.

Test połączenia z bazą MySQL

Po utworzeniu bazy i ustawieniu danych dostępowych można uruchomić test warstwy JDBC:

```
mvn compile exec:java "-Dexec.mainClass=cinema.db.DbTest"
```

Program powinien wyświetlić listę użytkowników oraz wyniki prób logowania. Brak połączenia oznacza zwykle nieuruchomiony serwer MySQL, błędne hasło lub brak bazy cinema_db.

Konta testowe

Login	Hasło	Rola
admin	admin123	Menadżer
kasjer1	pass1	Pracownik
jan.kowalski	haslo1	Klient
anna.nowak	haslo2	Klient
piotr.wisniewski	haslo3	Klient

Struktura plików projektu

